PHASE

10

User Mode and System Calls

Run programs in ring 3, each in its own address space, and let them ask the kernel for services through a single, carefully guarded door

Depends on	Phase 9: locks and wait queues; Phase 6: VMM that works on any page directory; Phase 3: GDT with ring 3 segments
Delivers	A TSS, per-process page directories that share the kernel half, a first jump into ring 3, a system call gate on `int 0x80` with a dispatch table, an ELF loader for programs passed as GRUB modules, a tiny user runtime (`crt0`, syscall wrappers), pointer validation for every user buffer, and faulty programs that get killed while the kernel survives.

1 Scope and motivation

Everything so far has run in ring 0, where any bug can corrupt anything. An operating system's core promise is **protection**: a buggy or malicious program can crash itself, but not the kernel and not other programs. That promise rests on three hardware mechanisms introduced earlier: privilege rings (Phase 3), paging with the U/S bit (Phase 6), and the interrupt mechanism as the only way back into ring 0 (Phase 3). This phase connects them.

This phase also defines the **system call interface**, the boundary between user programs and the kernel. Every file operation, every process operation and every byte of output from now on crosses it. Two rules established here hold for the rest of the project: every argument from user space is treated as hostile, and every user pointer is checked before the kernel touches it.

2 Background

2.1 Limited direct execution

OSTEP chapter 6 frames the problem exactly. For speed, user programs should run directly on the CPU. For safety, they must not be able to do everything. The answer is to run them in **user mode**, where privileged instructions (`cli`, `hlt`, `lgdt`, `out`, loading CR3) raise #GP and pages without the U/S bit are unreachable. When a program needs something only the kernel can do, it executes a **trap** instruction that enters the kernel at an address the kernel chose at boot. On x86 that is `int 0x80` through an IDT gate whose DPL is 3, the only gate user code is allowed to invoke. MOS §1.6 walks through the steps of a `read` call, from the library wrapper down to the kernel and back.

2.2 What the CPU does on a ring change

When an interrupt or trap arrives while the CPU is in ring 3, it can't keep using the user's stack: the user controls that memory, and ESP might point anywhere. So the CPU switches to a kernel stack first, and it finds that stack in the **task state segment** (TSS): fields `ss0` and `esp0`. Then it pushes the user's SS and ESP followed by the usual EFLAGS, CS and EIP (Figure 1); these are the `useresp` and `ss` fields reserved at the top of `struct regs` in Phase 3. `iret` reverses the whole thing. Kagan uses the TSS only for this; x86 hardware task switching is ignored. Because each task has its own kernel stack, the scheduler must update `esp0` on every context switch.

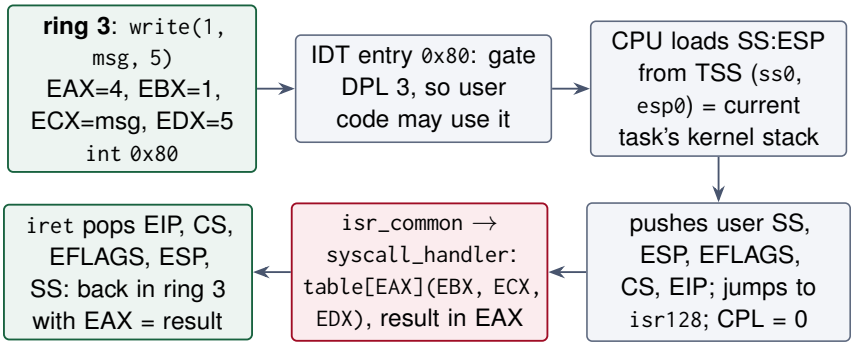


Figure 1: The round trip of a system call.

2.3 An address space per process

Each process gets its own page directory. Entries 0–767 (the user half) are private; entries 768–1023 are copied from the kernel's directory, and because every kernel page table is pre-allocated in Phase 6, those copies stay correct forever. Switching processes means loading the new directory into CR3. Figure 2 shows the layout of one process. User pages carry the U/S bit in both their PDE and PTE; kernel pages never do, so even though the kernel is mapped in every address space, user code can't touch it.

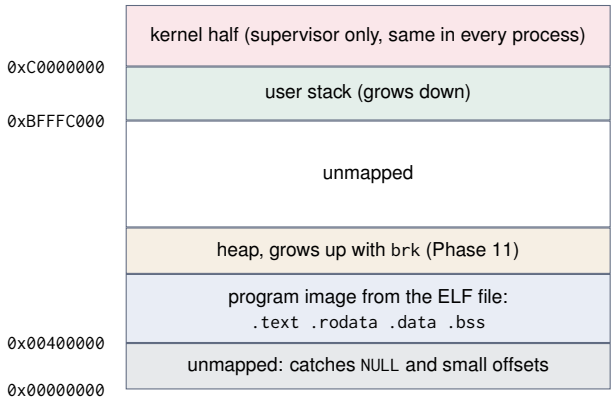


Figure 2: The address space of a Kagan user process.

2.4 Never trust a user pointer

When a program calls `write(1, buf, len)`, `buf` is just a number the program chose. It could point into the kernel (to trick the kernel into printing a secret), be unmapped (to crash the kernel), or wrap around the end of the address space. The kernel has rights the program lacks, and a program that gets the kernel to use those rights on its behalf has turned it into a **confused deputy**. Every user pointer must be checked before use: the whole range lies below `0xC0000000`, doesn't overflow, and every page in it is present and user-accessible, and also writable when the kernel writes to it. `copy_from_user` and `copy_to_user` do those checks and return `-EFAULT` instead of faulting. Check once and read once: if the kernel checks a value in user memory and then reads it again to use it, the program can change it in between (a **double fetch**), so the kernel copies user data in, then checks and uses only its copy. MOS §9.7 shows what happens to kernels that skip this.

2.5 Loading a program

A user program is an **ELF** executable. Its header gives the entry point and a table of **program headers**, and each `PT_LOAD` header says: take `p_filesz` bytes at offset `p_offset` in the file, place them at virtual address `p_vaddr`, and zero the rest up to `p_memsz` (that tail is `.bss`). The loader allocates and maps pages, copies, zeroes, sets up a user stack, and jumps. OSC §2.5 covers linkers and

loaders. Until Kagan has a file system (Phase 13), programs arrive as **Multiboot modules**: GRUB (or QEMU's `-initrd`) loads extra files into memory and lists them in the Multiboot information, just as LOB chapter 7 does.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	LOB	Ch. 7 *The Road to User Mode* (GRUB modules, executing a program); Ch. 11 *User Mode*; Ch. 13 *System Calls*	47–49, 65–68, 71–72	The x86 steps of Tasks 10.1–10.6.
PRIMARY	OSTEP	Ch. 6 §6.1–6.2 *Basic Technique, Restricted Operations*	49–55	User mode, kernel mode, traps and the trap table.
PRIMARY	MOS	§1.6 *System Calls* (process, file and directory calls; the steps of `read`)	50–62	What a system call interface looks like, and a model to copy.
SECONDARY	OSC	§2.1–2.5 *Services, Interface, System Calls, Linkers and Loaders*	55–77	Parameter passing, API versus system call, loaders.
SECONDARY	OSC	§1.4 *Operating-System Operations* (dual mode, timers)	21–27	Why the timer is part of protection.
SECONDARY	OSDI	§1.4 *System Calls*	26–42	The POSIX calls a UNIX needs, in one tour.
SECONDARY	OSDI	§2.7 *The System Task in MINIX 3*	192–204	How MINIX lets user-space servers request privileged work.
OPTIONAL	MOS	§12.2 *Interface Design*	985–993	Principles for choosing a system call set.
OPTIONAL	MOS	§9.7 *Exploiting Software*	639–657	Why every user pointer must be checked.
OPTIONAL	OSC	§17.3 *Protection Rings*	669–671	Rings in general, and why most OSes use only two.
PRIMARY	LKD	Ch. 5, *System Calls*	69–84	Numbers, the handler, parameter passing and verifying user pointers.
OPTIONAL	LKP	Ch. 6, *Viewing the user and kernel stacks*	281–289	Two stacks per thread, seen on a running system.

The Intel SDM Volume 3A is the reference for the TSS format (the task management chapter) and for the stack switch on a privilege change (the interrupt chapter), and the ELF specification (*Tool Interface Standard, ELF 1.2*) for the header formats.

Prior art: MINIX 3

In MINIX, a POSIX call like `read` is not a kernel call at all. The library turns it into a message, and the kernel's only job is to deliver that message to a user-space server (the file system or the process manager) through `sys_call` in `kernel/proc.c (07400)`. When a server needs something privileged, such as copying data between address spaces or writing an I/O port, it sends a *kernel call* to the **system task**, `kernel/system.c (09700)`, described in OSDI §2.7. So MINIX has two layers of calls where Kagan has one: an `int 0x80` goes straight to a kernel function.

Linux, for comparison. LKD Linux on i386 enters system calls through `int 0x80` (or `sysenter`) into a table indexed by the number in `EAX`, and every user pointer goes through `copy_from_user` and `copy_to_user`, exactly as in Kagan.

4 Implementation

Task 10.1 Install a TSS

Define the 104-byte TSS structure, put it in GDT entry 5 (selector `0x28`) with access byte `0x89` (present, ring 0, 32-bit available TSS) and byte granularity, and load it with `ltr`. Only `ss0` and `esp0` matter. Setting `iomap_base` to the structure's size means "no I/O permission bitmap", so any port access from ring 3 faults.

kernel/arch/i386/tss.c

```
struct tss {
    uint32_t prev_tss;
    uint32_t esp0;              /* kernel stack pointer loaded on entry from ring 3 */
    uint32_t ss0;               /* kernel stack segment: 0x10                       */
    uint32_t esp1, ss1, esp2, ss2;
    uint32_t cr3, eip, eflags, eax, ecx, edx, ebx, esp, ebp, esi, edi;
    uint32_t es, cs, ss, ds, fs, gs, ldt;
    uint16_t trap, iomap_base;
} __attribute__((packed));
_Static_assert(sizeof(struct tss) == 104, "TSS layout");

static struct tss tss;

void tss_init(void)
{
    tss.ss0 = 0x10;
    tss.iomap_base = sizeof(tss);
    gdt_set(5, (uint32_t)&tss, sizeof(tss) - 1, 0x89, 0x0);
    __asm__ volatile ("ltr %0" : : "r"((uint16_t)0x28));
}

void tss_set_kernel_stack(uint32_t esp0) { tss.esp0 = esp0; }
```

Verify: `info registers` in the QEMU monitor shows `TR=0028` with the TSS's base address.

Task 10.2 Give every process an address space

Add a page directory pointer to `struct task` (kernel threads use the kernel's directory). Write `vmm_create_address_space()`: take a page with `kalloc_page`, leave entries 0–767 empty, copy entries 768–1023 from the kernel directory. In `schedule()`, just before `switch_context`, load the next task's directory into CR3 if it differs from the current one, and call `tss_set_kernel_stack` with the top of the next task's kernel stack.

Verify: Two kernel threads with different directories both run; `info mem` shows the kernel mappings in each.

Task 10.3 The first jump into ring 3

Start with a minimal case before loading real programs. Create an address space, map one user page at `0x00400000` containing just the two bytes `EB FE` (`jmp $`, an infinite loop), map one user stack page, and enter ring 3 by building the frame `iret` expects: SS, ESP, EFLAGS (with IF set), CS, EIP. The user selectors have RPL 3: `0x1B` for code and `0x23` for data.

kernel/arch/i386/usermode.s

```
; void enter_user_mode(uint32_t entry, uint32_t user_esp)   -- never returns
global enter_user_mode
enter_user_mode:
    mov  ecx, [esp + 4]     ; user entry point
    mov  edx, [esp + 8]     ; user stack pointer
    mov  ax, 0x23           ; user data selector (GDT index 4, RPL 3)
    mov  ds, ax
    mov  es, ax
    mov  fs, ax
    mov  gs, ax
    push dword 0x23         ; SS
    push edx                ; ESP
    pushfd
    pop  eax
    or   eax, 0x200         ; IF = 1: interrupts stay on in user mode
    push eax                ; EFLAGS
    push dword 0x1B         ; CS (GDT index 3, RPL 3)
    push ecx                ; EIP
    iret                    ; drop to ring 3
```

Verify: The monitor stays responsive (the timer preempts the looping user task), and when you stop QEMU while that task runs, `info registers` shows `CS=001b` and `EIP=00400000`.

Task 10.4 The system call gate

Add stub 128 to `isr.s` and install it with `idt_set_gate(0x80, isr128, 0x08, 0xEF)`: a 32-bit *trap* gate with DPL 3. DPL 3 lets ring 3 use it; the trap type leaves interrupts enabled during system calls, so a long call can be preempted like any other kernel code (the Phase 9 locks make that safe). Register a handler that dispatches on EAX. Use the Linux i386 numbers for the calls you implement; then any Linux reference table matches Kagan.

kernel/syscall/syscall.c

```
enum { SYS_exit = 1, SYS_read = 3, SYS_write = 4, SYS_getpid = 20,
       SYS_sched_yield = 158, SYS_nanosleep = 162, NR_SYSCALLS = 256 };

typedef int32_t (*syscall_fn)(uint32_t a, uint32_t b, uint32_t c);

static const syscall_fn syscall_table[NR_SYSCALLS] = {
    [SYS_exit]        = sys_exit,
    [SYS_read]        = sys_read,
    [SYS_write]       = sys_write,
    [SYS_getpid]      = sys_getpid,
    [SYS_sched_yield] = sys_yield,
    [SYS_nanosleep]   = sys_sleep_ms,      /* simplified: argument is milliseconds */
};

static void syscall_handler(struct regs *r)
{
    if (r->eax >= NR_SYSCALLS || !syscall_table[r->eax]) {
        r->eax = (uint32_t)-ENOSYS;
        return;
    }
    r->eax = syscall_table[r->eax](r->ebx, r->ecx, r->edx);   /* return value to user */
}
```

Portability note

`syscall_handler` is generic code, yet it reads x86 registers by name. Dispatch through three one-line inline helpers in the x86 header instead, such as `tf_sysno(r)`, `tf_arg(r, n)` and `tf_set_ret(r, v)`: Phase 16 makes exactly these part of the interface every architecture provides. The ports keep these Linux i386 call numbers and change only the trap: RISC-V uses `ecall` with the number in `a7` and the arguments in `a0`–`a5`, AArch64 uses `svc #0` with the number in `x8` and the arguments in `x0`–`x5`, and the result comes back in `a0` or `x0`.

Task 10.5 A user runtime

Create `user/` with its own linker script (programs start at `0x00400000`), a start file that calls `main` and passes its return value to `exit`, and syscall wrappers in inline assembly. Build each program as a separate ELF with the same cross compiler, linked with `-nostdlib` and `-T user/user.ld`.

user/crt0.s

```
global _start
extern main
extern exit
section .text
_start:
    call main               ; argc and argv arrive in Phase 11
    push eax
    call exit               ; exit(main's return value), never returns
```

user/lib/syscall.c

```
static inline int32_t syscall3(uint32_t n, uint32_t a, uint32_t b, uint32_t c)
{
    int32_t ret;
    __asm__ volatile ("int $0x80" : "=a"(ret) : "a"(n), "b"(a), "c"(b), "d"(c) : "memory");
    return ret;
}

int  write(int fd, const void *buf, unsigned len) { return syscall3(4, fd, (uint32_t)buf, len); }
int  getpid(void)                                 { return syscall3(20, 0, 0, 0); }
void exit(int code)                               { syscall3(1, code, 0, 0); for (;;) { } }
```

Task 10.6 Load programs as boot modules

Add `module /boot/hello.elf` lines to `grub.cfg`. For quick runs, QEMU's Multiboot loader takes the modules as a comma-separated `-initrd` list:

terminal

```
qemu-system-i386 -kernel build/kernel.elf -serial stdio \
    -initrd "build/user/hello.elf,build/user/evil.elf"
```

In the kernel, walk `mods_count` and `mods_addr` (through `P2V`), reserve the modules' frames in the PMM (as in Phase 5), and keep a table of name, start and end for the loader.

Task 10.7 The ELF loader

Write `elf_load(pgdir, image, size, &entry)`. The image comes from outside the kernel, so validate everything before trusting anything, in a separate function `elf_check` that only reads the bytes. The header needs the magic bytes `7F 45 4C 46`, 32-bit class, little-endian, type `ET_EXEC` (2), machine `EM_386` (3), `e_phentsize` equal to the size of `struct elf32_phdr`, and between 1 and 16 program headers that lie inside the file. Each `PT_LOAD` header must then pass five checks, with every sum checked for overflow. Its file bytes, `p_offset` plus `p_filesz`, lie inside the image; otherwise the copy

reads past it. `p_filesz` is at most `p_memsz`; otherwise the copy writes past the frames allocated for `p_memsz`. `p_vaddr` is at least `USER_BASE` (`0x00400000`, where user programs are linked). `p_vaddr` plus `p_memsz` is at most `0xC0000000`. The segments come in address order and never share a page. Last, the entry point must lie inside an executable segment. The minimum address matters more than it looks: a program that maps page 0 turns every kernel `NULL` dereference into a read of memory it controls, and on a CPU without SMEP even into running its code.

Only then does `elf_load` allocate and map user pages (writable only if the segment's flags say so), copy `p_filesz` bytes through the direct map, and zero the rest up to `p_memsz`. Take every frame that goes to user space from `kalloc_page`, which zeroes it (Phase 7), so no process ever sees what an earlier one left in a frame, not even in the bytes after `p_filesz` in a segment's last page. Then map a user stack below `0xC0000000`, create a task that runs `enter_user_mode(entry, stack_top)`, and make it ready.

include/elf.h

```
struct elf32_ehdr {
    uint8_t  e_ident[16];          /* 0x7F 'E' 'L' 'F', class, data, version ... */
    uint16_t e_type, e_machine;
    uint32_t e_version, e_entry, e_phoff, e_shoff, e_flags;
    uint16_t e_ehsize, e_phentsize, e_phnum, e_shentsize, e_shnum, e_shstrndx;
} __attribute__((packed));

struct elf32_phdr {
    uint32_t p_type;               /* 1 = PT_LOAD                  */
    uint32_t p_offset, p_vaddr, p_paddr;
    uint32_t p_filesz, p_memsz;    /* memsz > filesz: zero the tail */
    uint32_t p_flags, p_align;     /* flags: 1 = X, 2 = W, 4 = R    */
} __attribute__((packed));
```

kernel/exec/elf_check.c

```
#define USER_BASE 0x00400000u      /* nothing below: page 0 is never mapped */
#define MAX_PHNUM 16

/* Reads only the bytes it is given: no allocation, no mapping, so it builds on the host too. */
int elf_check(const uint8_t *image, size_t size, uint32_t *entry)
{
    const struct elf32_ehdr *eh = (const void *)image;
    uint32_t phend, end, prev_end = 0;
    bool entry_ok = false;

    if (size < sizeof *eh || memcmp(eh->e_ident, "\x7F" "ELF", 4) != 0 ||
        eh->e_ident[4] != 1 || eh->e_ident[5] != 1 ||          /* 32-bit, little-endian */
        eh->e_type != 2 || eh->e_machine != 3 ||
        eh->e_phentsize != sizeof(struct elf32_phdr) ||
        eh->e_phnum == 0 || eh->e_phnum > MAX_PHNUM ||
        __builtin_add_overflow(eh->e_phoff, eh->e_phnum * sizeof(struct elf32_phdr), &phend) ||
        phend > size)
        return -ENOEXEC;

    const struct elf32_phdr *ph = (const void *)(image + eh->e_phoff);
    for (int i = 0; i < eh->e_phnum; i++, ph++) {
        if (ph->p_type != 1)                                   /* only PT_LOAD is loaded */
            continue;
        if (__builtin_add_overflow(ph->p_offset, ph->p_filesz, &end) || end > size ||
            ph->p_filesz > ph->p_memsz || ph->p_vaddr < USER_BASE ||
            __builtin_add_overflow(ph->p_vaddr, ph->p_memsz, &end) || end > KERNEL_VMA ||
            (ph->p_vaddr & ~0xFFFu) < prev_end)                /* in order, no shared page */
            return -ENOEXEC;
        prev_end = (end + 0xFFFu) & ~0xFFFu;
        if ((ph->p_flags & 1) && eh->e_entry >= ph->p_vaddr && eh->e_entry < end)
```

```
            entry_ok = true;                                    /* entry in an X segment */
    }
    if (!entry_ok)
        return -ENOEXEC;
    *entry = eh->e_entry;
    return 0;
}
```

Because `elf_check` is a pure function of a byte buffer, it also compiles on the host, and that makes it a perfect **fuzz target**. A coverage-guided fuzzer such as libFuzzer (part of clang) calls it millions of times with mutated inputs under the address and undefined-behaviour sanitizers, and stops at the first out-of-bounds read or overflow. Seed it with the real programs, so it starts from valid files and mutates them into almost-valid ones.

tests/host/fuzz_elf.c

```
/* clang -g -O1 -fsanitize=fuzzer,address,undefined -Iinclude \
 *       tests/host/fuzz_elf.c kernel/exec/elf_check.c -o build/fuzz_elf
 * mkdir -p corpus && cp build/user/*.elf corpus/ && build/fuzz_elf -max_total_time=600 corpus */
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
    uint32_t entry;
    elf_check(data, size, &entry);
    return 0;
}
```

Verify: A `hello` program prints “Merhaba from ring 3, pid 3” through `write`, then exits, and its pages go back to the PMM (free count restored). The ELF fuzz target runs for 10 minutes without a sanitizer report.

Task 10.8 Guard the door

Write `copy_from_user`, `copy_to_user` and `strncpy_from_user` in `kernel/syscall/uaccess.c`, and make that file the only place in the kernel that touches user memory. A user address arrives as a `uint32_t` system call argument: keep it a `uintptr_t` with a name starting with `u`, never cast it to a pointer anywhere else, and hand it to these functions. Reading needs present user pages; writing also needs `PG_WRITE`. That second check is not a detail. The ELF loader maps text read-only and `CR0.WP` is set, so without it a `copy_to_user` into a program’s own code faults in ring 0, and any program could panic the kernel with `read(0, (void *)main, 16)`. Phase 11 extends the checks to pages that don’t exist yet (a lazily grown heap, copy-on-write pages), and because only `uaccess.c` touches user memory, that is a change in one file. `sys_write` copies the user buffer in chunks into a kernel buffer before printing. A `sys_read` that waits for input checks its buffer with `user_range_ok` before it blocks, as Linux’s `access_ok` does, so a bad pointer fails at once; `copy_to_user` checks again when the data is there.

kernel/syscall/uaccess.c

```
/* The only file in the kernel that dereferences a user address. */
bool user_range_ok(uintptr_t ustart, size_t n, bool write)
{
    uintptr_t end;
    if (__builtin_add_overflow(ustart, n, &end) || end > KERNEL_VMA)
        return false;                                   /* wraps around, or reaches the kernel */
    uint32_t need = PG_PRESENT | PG_USER | (write ? PG_WRITE : 0);
    for (uintptr_t pg = ustart & ~0xFFFu; pg < end; pg += PAGE_SIZE)
        if (!vmm_user_page_ok(current->pgdir, pg, need))  /* all bits set in PDE and PTE */
            return false;
    return true;
}
```

```
int copy_from_user(void *dst, uintptr_t usrc, size_t n)
{
    if (!user_range_ok(usrc, n, false))
        return -EFAULT;
    memcpy(dst, (const void *)usrc, n);                /* current address space is active */
    return 0;
}

int copy_to_user(uintptr_t udst, const void *src, size_t n)
{
    if (!user_range_ok(udst, n, true))                 /* read-only text: -EFAULT, not a panic */
        return -EFAULT;
    memcpy((void *)udst, src, n);
    return 0;
}
```

Two more rules keep the boundary tight. **Copy once**: read a structure or a string from user space into the kernel exactly once, then check and use only the copy, never the user memory again. **Zero before copy**: set every structure that goes back to user space to zero with `memset` before you fill it, so padding bytes never carry old kernel stack or heap contents out. Then write down each call's contract in `docs/security/syscalls.md`, one row per system call: its arguments, how each is validated, which sums are checked for overflow, whether input is copied once, and which errors it returns. A new system call is not done until its row exists, and later phases add their rows the same way.

docs/security/syscalls.md

```
| Call   | Arguments    | Validation                     | Overflow  | Copy-once      | Errors        |
|--------|--------------|--------------------------------|-----------|----------------|---------------|
| write  | fd, buf, len | fd is 1 or 2; buf readable     | buf + len | 256-byte chunks| EBADF, EFAULT |
| read   | fd, buf, len | fd is 0; buf writable, checked | buf + len | output only    | EBADF, EFAULT |
|        |              | before blocking                |           |                |               |
| getpid | none         | none                           | none      | none           | none          |
```

Then make exceptions depend on who caused them: if `(r->cs & 3) == 3`, the fault came from user mode, so print one line such as "hello (pid 3): SIGSEGV at 0x00000000" and kill that task; only faults in ring 0 still panic. Print the name, the PID, the signal and the faulting address, and nothing else. The register dump of Phase 3 is a kernel diagnostic: kernel addresses and stack contents help an attacker more than they help the user.

Task 10.9 Write evil programs

A protection mechanism is only as good as its tests. Write user programs that each try one thing they must not do, load them all at once, and check that each is killed, gets an error or is refused by the loader while `hello` and the kernel carry on. `map_zero` and `badelf` are not written in C: a short host script patches a copy of `hello.elf` (a segment moved to address 0, a `p_filesz` larger than `p_memsz`).

Program	What it tries	Expected result
`divzero`	divide by zero	killed (#DE from ring 3)
`nullptr`	read address 0	killed (page fault, user mode)
`kpeek`	read `*(int *)0xC0100000`	killed (protection violation)
`cli`	execute `cli` or `hlt`	killed (#GP)
`badptr`	`write(1, (void *)0xC0000000, 16)`	`write` returns `-EFAULT`
`hugelen`	`write(1, buf, 0xFFFFFFFF)`	`-EFAULT` (range wraps around)
`badsys`	`int 0x80` with EAX = 9999	returns `-ENOSYS`
`ro_read`	`read(0, (void *)main, 16)`, into its own read-only text	`-EFAULT`, kernel survives
`map_zero`	an ELF file with a segment at address 0	refused (`-ENOEXEC`)
`badelf`	an ELF file whose `p_filesz` is larger than `p_memsz`	refused (`-ENOEXEC`)
`stacksmash`	a test-only system call that overflows a local kernel buffer	panic: stack smashing detected

Verify: The first ten behave as listed in a single boot under `make test`, and `ps` afterwards shows only the kernel's own tasks. `stacksmash` exists only in test kernels and is the one program whose expected result is a panic (Task 10.10): `make test` runs it last, in a boot of its own, and counts the "stack smashing detected" panic as a pass.

Task 10.10 Threat model, stack protector and static analysis

Ring 3 exists now, so the Phase 0 stub `docs/security/threat-model.md` becomes version 0 of a real threat model. Keep its assets, adversaries and out-of-scope list, and add two things. First, the trust boundaries that exist now: ring 3 to ring 0 (`int 0x80` and every exception raised in user mode), and ELF files to the loader. Second, walk each boundary with **STRIDE**, which names six kinds of threat: spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege. Write one line per threat: what could happen, which kernel control stops it, and how much risk is left. Record the accepted risks as well, such as the lack of a no-execute bit for user data under 32-bit paging without PAE. Phases 13 and 15 add their boundaries to the same file.

docs/security/threat-model.md (v0, added sections)

```
## Trust boundaries (v0)
B1  ring 3 -> ring 0: int 0x80, and every exception raised in user mode
B2  ELF image -> loader (boot modules now, files from Phase 13)

| ID  | STRIDE | Threat                                  | At | Control                       | Left |
|-----|--------|-----------------------------------------|----|-------------------------------|------|
| T1  | E      | kernel uses a user pointer directly     | B1 | only uaccess.c; CI grep       | low  |
| T2  | D      | copy_to_user into read-only page        | B1 | PG_WRITE check; ro_read       | low  |
| T3  | T/E    | length overflow, out-of-bounds write    | B1 | overflow checks; UBSan build | low  |
| T4  | E      | user memory read twice (TOCTOU)         | B1 | copy once, use the copy       | low  |
| T5  | I      | padding leaks kernel bytes              | B1 | zero before copy              | low  |
| T6  | I      | old frame contents reach a process      | B1 | frames zeroed on allocation   | low  |
| T7  | I/T    | FPU registers shared between tasks      | B1 | fxsave/fxrstor per task       | low  |
| T8  | E      | user context restored with ring 0 bits | B1 | kernel sets CS, SS, EFLAGS    | low  |
| T9  | E      | stack buffer overflow in the kernel     | B1 | -fstack-protector-strong      | low  |
| T10 | T/E    | malformed ELF: OOB access, page 0       | B2 | elf_check; host fuzz target   | low  |

Accepted: no NX for user data (32-bit paging without PAE has no NX bit);
canary from rdtsc when rdrand is missing (predictable).
```

Next, arm the stack protector. Compile the kernel with `-fstack-protector-strong`: GCC then places a canary between the local arrays and the saved return address of every function that has such arrays, and checks it before the function returns. On `i686-elf`, GCC reads the canary from a global variable, `__stack_chk_guard` (there is no thread-local storage), and calls `__stack_chk_fail` when the check fails. The kernel provides both, and the second only needs `panic`. The guard must be unpredictable, and

setting it has one trap. A protected function that was entered before the guard changed compares its old canary with the new guard when it returns, and reports stack smashing that never happened. So set the guard in `canary_init`, which is built with `-fno-stack-protector`, and call it first thing in `kmain`, which gets the same flag and never returns. Use `rdrand` when CPUID reports it (QEMU's `-cpu max` has it); otherwise mix `rdtsc` with the Multiboot memory map. That fallback is predictable, which is acceptable for a non-production kernel and nothing more, so write it into the threat model.

kernel/security/canary.c (built with -fno-stack-protector)

```
uint32_t __stack_chk_guard;

__attribute__((noreturn)) void __stack_chk_fail(void)
{
    panic("stack smashing detected");
}

static bool rdrand32(uint32_t *v)
{
    for (int i = 0; i < 10; i++) {                  /* rdrand may fail briefly: retry */
        uint8_t ok;
        __asm__ volatile ("rdrand %0; setc %1" : "=r"(*v), "=qm"(ok));
        if (ok)
            return true;
    }
    return false;
}

void canary_init(const struct multiboot_info *mbi)  /* first call in kmain */
{
    uint32_t a, b, c, d, g = 0;
    __asm__ volatile ("cpuid" : "=a"(a), "=b"(b), "=c"(c), "=d"(d) : "a"(1));
    if (!(c & (1u << 30)) || !rdrand32(&g)) {       /* no rdrand: weak, documented fallback */
        uint32_t lo, hi;
        __asm__ volatile ("rdtsc" : "=a"(lo), "=d"(hi));
        g = lo ^ hi;
        const uint32_t *m = P2V(mbi->mmap_addr);
        for (uint32_t i = 0; i < mbi->mmap_length / 4; i++)
            g = (g ^ m[i]) * 0x01000193u;           /* FNV-style mixing */
    }
    __stack_chk_guard = g;
}
```

Makefile (stack protector)

```
CFLAGS += -fstack-protector-strong
$(BUILD)/kernel/kmain.o $(BUILD)/kernel/security/canary.o: CFLAGS += -fno-stack-protector
```

Last, add static analysis to the CI workflow of Phase 7. `cppcheck` understands GCC extensions and reports few false positives. Run it with `--enable=warning,portability` and let any finding fail the job. When a finding is wrong, suppress that one finding in `tools/cppcheck.supp` with a comment that says why, never a whole check. One more step enforces the `uaccess.c` rule: outside that file, no `u`-named user address is ever cast to a pointer.

.github/workflows/ci.yml (additional steps)

```
      - run: sudo apt-get install -y cppcheck
      - run: cppcheck --enable=warning,portability --error-exitcode=1
             --suppressions-list=tools/cppcheck.supp -Iinclude kernel lib
      - run: |
          ! grep -rnE '\*\) *u[a-z_]+' kernel --include='*.c' --exclude=uaccess.c
```

Verify: The threat model lists both boundaries, with a STRIDE line, a control and a residual risk for each threat. With the protector on, every test passes and `stacksmash` ends in "stack smashing detected". `cppcheck` and the grep step pass in CI, and a cast of a user address added on purpose outside `uaccess.c` fails the job.

5 Testing and debugging

Symptom	Likely cause	Resolution
#GP on the `iret` into ring 3	Selectors without RPL 3, or GDT entries 3–4 with the wrong DPL	Use `0x1B` and `0x23`; check access bytes `0xFA` and `0xF2`.
Page fault at the user entry point with error code 5 or 7 (user mode, protection violation)	The PDE or the PTE lacks the U/S bit	Both levels need it; print both entries for the faulting address.
Triple fault on the first interrupt or syscall from ring 3	TSS not loaded, `esp0` zero, or the TSS descriptor is wrong	Check `TR` in `info registers`; set `esp0` before entering ring 3.
User program never gets preempted	EFLAGS pushed for `iret` without IF	OR `0x200` into EFLAGS in `enter_user_mode`.
Second process crashes after a context switch	CR3 switched, but `esp0` still points at the first task's stack	Update `esp0` and CR3 together, on every switch.
Kernel crashes inside a system call on a bad pointer	A pointer was used directly instead of through `copy_from_user`	Search for every user pointer dereference; there should be none outside `uaccess.c`.
Ring-0 page fault with error code 3 inside `copy_to_user`	The destination page is present but read-only, and only presence and U/S were checked	Require `PG_WRITE` for every page a write touches.
"stack smashing detected" right after boot, with no overflow in sight	The guard changed while a protected function was still running, so its return compared against the new value	Set the guard in a `-fno-stack-protector` function called first in `kmain`.

6 Exit criteria

Exit criteria: Protection works

- ☐ The TSS is loaded, and `esp0` and CR3 are updated on every context switch.
- ☐ Each process has its own page directory sharing the kernel half.
- ☐ ELF programs from boot modules run in ring 3 and use `write`, `getpid`, `sleep` and `exit`.
- ☐ Only `uaccess.c` touches user memory, and `copy_to_user` requires writable pages.
- ☐ The ELF loader passes every check of Task 10.7, and its fuzz target runs 10 minutes clean.
- ☐ All eleven evil programs behave as listed; apart from `stacksmash`'s expected panic, user code never brings the kernel down.
- ☐ `docs/syscalls.md` lists each call: number, arguments, return value, errors; `docs/security/syscalls.md` has its boundary row.
- ☐ The threat model is at version 0, the stack protector is on, and CI runs `cppcheck` and the uaccess grep.

Review questions

1. List, in order, everything the CPU does when a ring-3 program executes `int 0x80`.
2. Why does the TSS need `esp0`, and why must it change on every context switch?
3. Why is the system call gate a trap gate with DPL 3, while exception gates are interrupt gates with DPL 0?
4. How does a system call's return value get back to the user program?
5. Why must both the PDE and the PTE have the U/S bit set? What happens if only the PTE has it?
6. What does `copy_from_user` check, and what could a program do if it didn't?
7. The kernel is mapped in every address space. Why can't user code read it?
8. Why must `copy_to_user` check the R/W bit itself, when the CPU checks it anyway?

7 Extensions

Replace `int 0x80` with the faster `sysenter/sysexit` pair and measure the difference with `rdtsc`. Put `argc`, `argv` and the environment on the initial user stack in the System V i386 layout, so `main(int argc, char **argv)` works (Phase 11 needs this for `exec` anyway). Add a read-only page shared by all processes that holds the current tick count, so programs can read the time without a system call; Linux's vDSO grew from the same idea.

Next: **Phase 11 Process API: fork, exec, wait and exit** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and OSTEP are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.